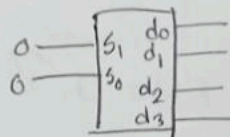


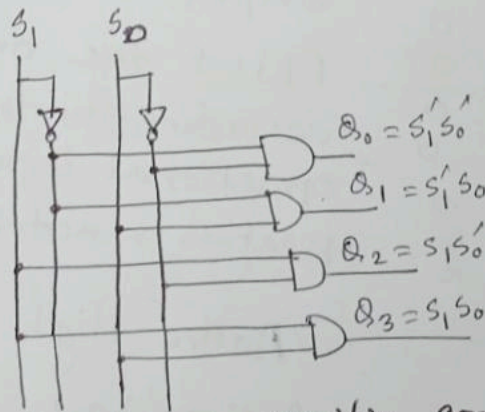
Decoder

A decoder is a circuit that changes a code into a set of signals. It is called decoder since it does the reverse operation of encodes. So, In general what do we mean by decoding. Yes! Elaborating or breaking a code- which is in a normally short form. We decode and expand it's meaning. Here in electronic circuit decoder also ~~split~~^{convert} binary combination into specific set of signals.

A decoder has n inputs and 2^n outputs. less input more output.
So, If a decoder has 2 inputs it will have 4 outputs.

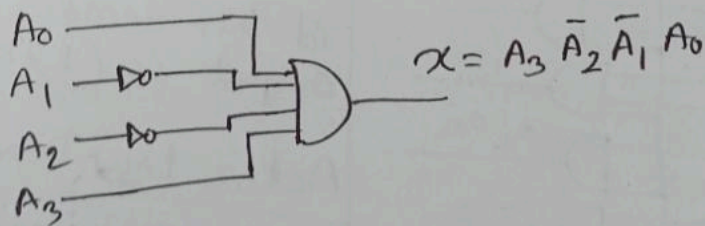


s_1	s_0	Q_0	Q_1	Q_2	Q_3	Q_0	Q_1	Q_2	Q_3
0	0	1	0	0	0	$s_1' s_0'$			
0	1	0	1	0	0		$s_1' s_0$		
1	0	0	0	1	0			$s_1 s_0'$	
1	1	0	0	0	1				$s_1 s_0$



We can also decode a specific combination. The design consideration is that we have to make the output high for that specific combination.

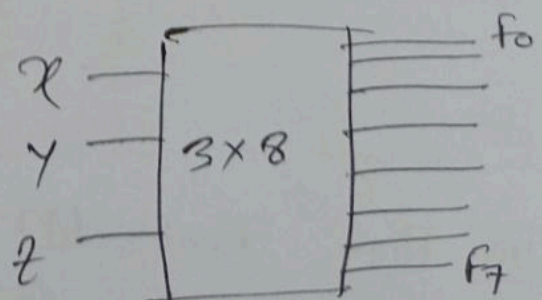
$A_0 = 1 \rightarrow A_0$
 $A_1 = 0 \rightarrow A_1'$
 $A_2 = 0 \rightarrow A_2'$
 $A_3 = 1 \rightarrow 1$



Similarly we can make 3-8 bit decoder

x	y	z	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	$x'y'z'$
0	0	0	1	0	0	0	0	0	0	0	$x'y'z'$
0	0	1	0	1	0	0	0	0	0	0	$x'y'z$
0	1	0	0	0	1	0	0	0	0	0	$x'y z'$
0	1	1	0	0	0	1	0	0	0	0	$x'y z$
1	0	0	0	0	0	0	1	0	0	0	$x y' z'$
1	0	1	0	0	0	0	0	1	0	0	$x y' z$
1	1	0	0	0	0	0	0	0	1	0	$x y z'$
1	1	1	0	0	0	0	0	0	0	1	$x y z$

So, we can construct the circuit accordingly.



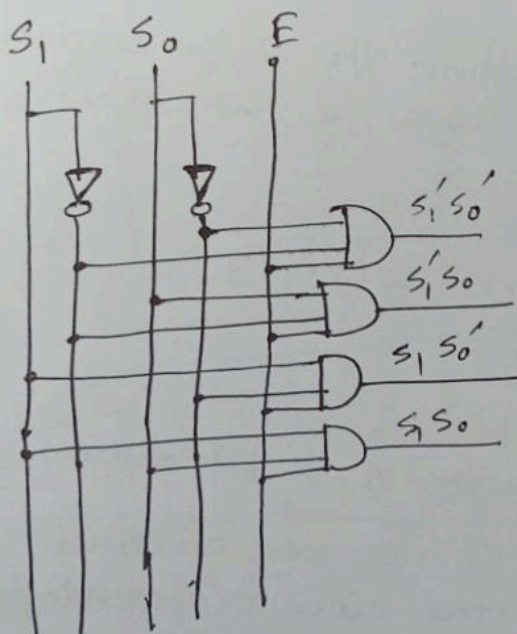
Enable

No we may want to control our decoder device through a mechanism. Whenever we want the operation of that decoder we can use that control to 'on' the decoder to operate or off the operation. Note this is not the power switch on and off. It is the on and off of the operation. This is an additional input, known as Enable.

Since we have used the and gate to construct the decoder if we fed a '0' the whole output of that gate (AND) will become '0'. Or if we fed '1' the output will be according to our actual inputs.

EN	S_1	S_0	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

EN = 0, deactivates the Decoder. all off the output will be zero.
 EN = 1 the output will be according to the desired operational input means operation is activated.



Active High Decoder Active Low Decoder

Active High Decoder: When the output of the decoder's active operational output is (1) high it is active high.

Active Low Decoder: When the output of the Decoder's active operational output is (0) Low is called active 'low'.

Active low and Active High can be defined with the relation of Enable input also.

Active High: Enable (1) → Output → (1)

Enable (0) → Output → (0) Inverted

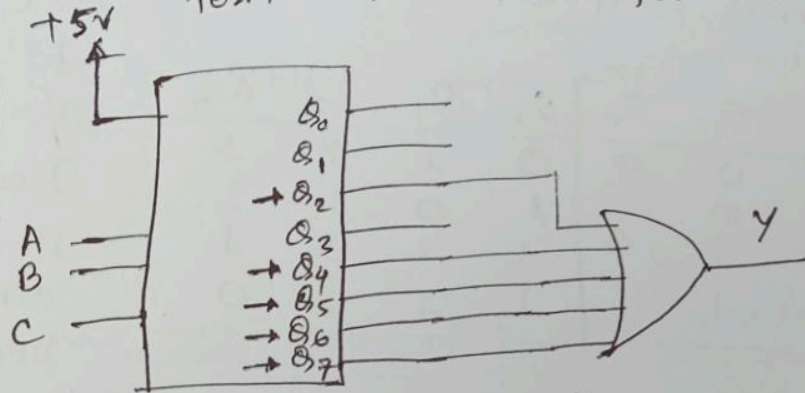
Active High: Works at EN 1; operational output 1 | Active low: Works at EN (0) operational o/p (0).

Since, active high decoders are min term generators we can implement any SOP expression using active high Decoders. We will need an or gate extra. How? Let's see.

	A	B	C	Y	minterms
0	0	0	0	0	
1	0	0	1	0	
2	0	1	0	1	$A'Bc'$
3	0	1	1	0	
4	1	0	0	1	ABc'
5	1	0	1	1	$AB'C$
6	1	1	0	1	ABC'
7	1	1	1	0	ABC

$$Y = A'Bc' + ABc' + AB'C + ABC' + ABC$$

(2)nd term 4th term 5th term 6th term 7th term



find which are the terms are from the equation. Then take those output terminals of the decoder and add.

$$Y(ABC) = \sum(2, 4, 5, 6, 7)$$

We can also design an adder using the decoder

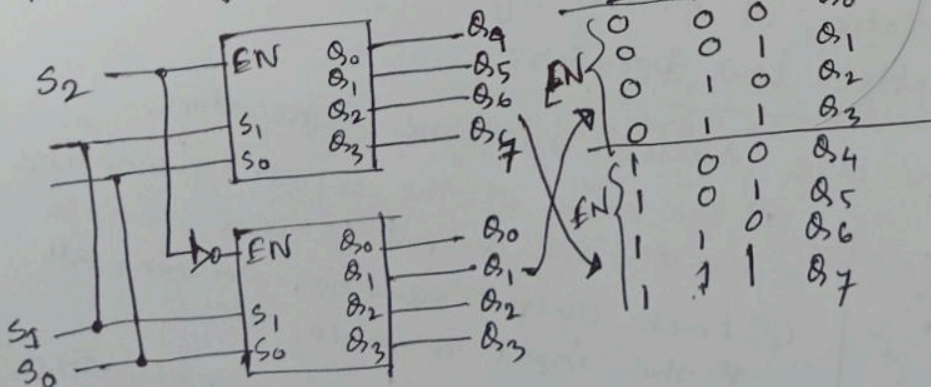
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$C(X, Y, Z) = \sum(3, 5, 6, 7)$$

$$S(X, Y, Z) = \sum(1, 2, 4, 7)$$

Decoder expansion

We can exploit the enable input to use small decoders to make larger decoders.



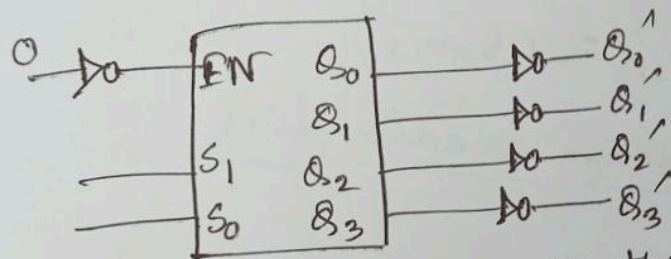
Use the MSB bit in the Enable with an inverter.

Active low Decoder Design:

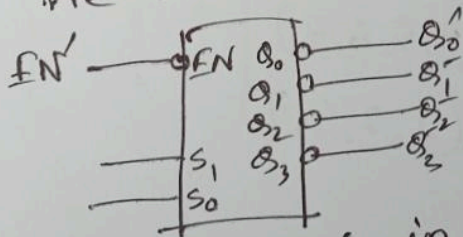
Since we have designed active High decoders already, we can use the same logic to design active low decoders. We just need to invert the EN input and invert the operational outputs.

EN	S ₁	S ₀	Q ₀	Q ₁	Q ₂	Q ₃
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

just inverting the opp of the active high Decoder.
We can use the whole circuit of active high Decoder. just invert the individual outputs and the Enable input



we can rearrange the block diagram as below.



See, If we put 1 in the enable input it will go inside as 0 and finally produce all output as zero at inside then 1 in all output's externally which is not the desired operation. Check with previously designed decoder. If we put 0 to EN then it will work properly as designed for active low operation.

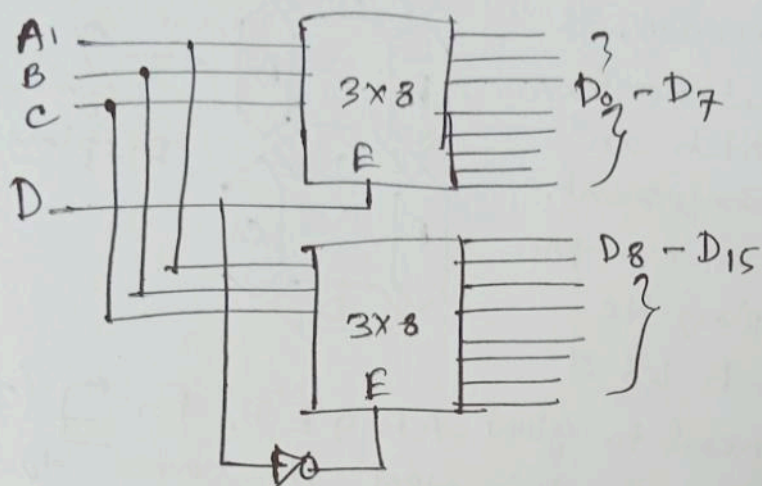
Relation of Active High and Active Low Decoders

Active High decoders are called minterm generators.

S ₁	S ₀	Q ₀	Q ₁	Q ₂	Q ₃	S ₁ ' S ₀ '
0	0	1	0	0	0	S ₁ ' S ₀ '
0	1	0	1	0	0	S ₁ ' S ₀
1	0	0	0	1	0	S ₁ S ₀ '
1	1	0	0	0	1	S ₁ S ₀

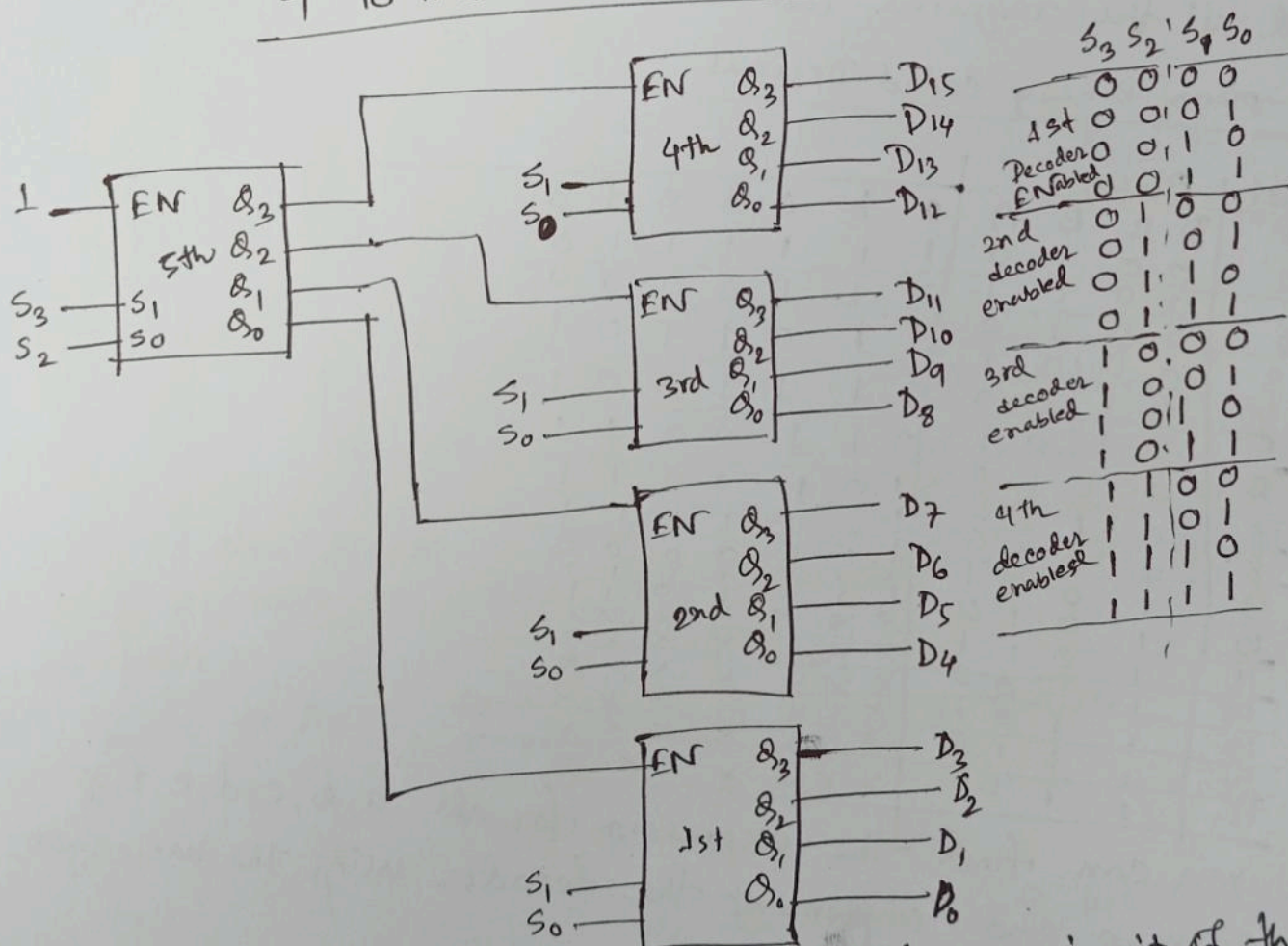
- ① For each of the input combinations exactly one output is true.
- ② Each output equation contains all of the input variables
- ③ It can be observed for all size of decoders.

3 to 8 decoder for 4 to 16 Decoders



First 8 Combination
the lower decoder will
be inactive, for last 8
combination ~~will be~~
upper decoder will be
inactive.

4 to 16 Decoder Using only 2-4 Decoder

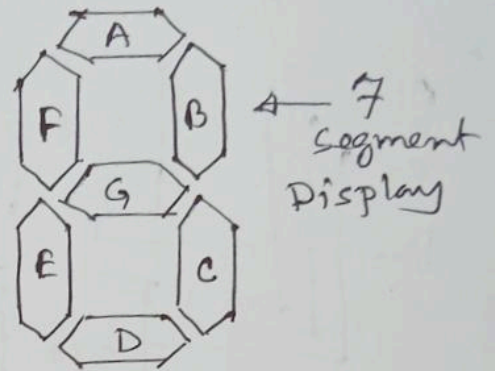


The output of 5th Decoder acts as the driver circuit of the other 4 decoders.

BCD To 7 Segment Decoder

How to design this.

This is a different type of decoder. It is not the same decoder what we have made so far. We need to switch on different segments or the arrangement. Each Boxes are leds. So, as an example to show a zero in the display we need to on A, B, C, D, E, and F led of the display. For '2' (2) we need to glow A B G E D. For '3' (3) we need to glow A, B, G, C, D. In this way we have to find all the required led



BCD - mean Binary coded Decimal

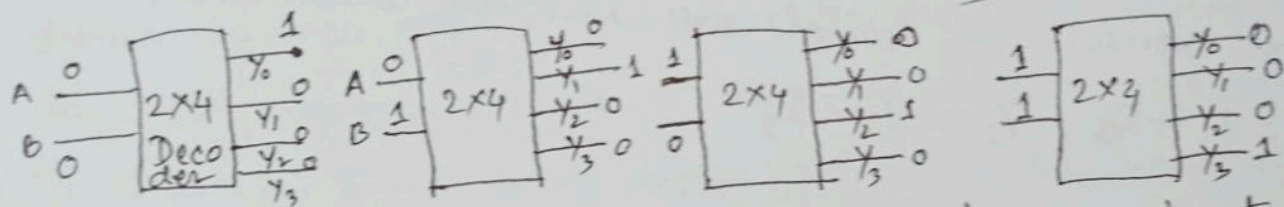
Decimal	Binary				a	b	c	d	e	f	g
0	D	C	B	A	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	0	0	1	1
6	0	1	1	0	1	0	1	1	0	1	1
7	0	1	1	1	1	0	1	1	1	1	1
8	1	0	0	0	1	1	1	0	0	0	0
9	1	0	0	1	1	1	1	0	0	1	1
10	1	0	1	0	x	x	x	x	x	x	x
11	1	0	1	1	x	x	x	x	x	x	x
12	1	1	0	0	x	x	x	x	x	x	x
13	1	1	0	1	x	x	x	x	x	x	x
14	1	1	1	0	x	x	x	x	x	x	x
15	1	1	1	1	x	x	x	x	x	x	x

Now, you can find the expression for all a, b, c, d, e, f, g using k-map and implement the decoder using the basic gates. Not required for now.

Encoder

Dr. Gaur

Encoder operates just reverse of the decoder. let's take an example that in Decoder we gave binary equivalent combination in the Inputs and got the decimal output (take as an example) in the output pins serially. [actually the combination, 1st, second, 3rd, 4th]



Encoder will do just reverse. If we give the input combination it will give the binary output.

For Decoder

	A	B	Y ₀	Y ₁	Y ₂	Y ₃
1st	0	0	1	0	0	0
2nd	0	1	0	1	0	0
3rd	1	0	0	0	1	0
4th	1	1	0	0	0	1

For encoder it will do just reverse.

Y ₀	Y ₁	Y ₂	Y ₃	A	B	
1	0	0	0	0	0	1st
0	1	0	0	0	1	2nd
0	0	1	0	1	0	3rd
0	0	0	1	1	1	4th

How to design Encoders

lets say a 4 to 2 bit Encoder

	inputs				outputs	
	I ₃	I ₂	I ₁	I ₀	Y ₁	Y ₀
1st	0	0	0	1	0	0
2nd	0	0	1	0	0	1
3rd	0	1	0	0	1	0
4th	1	0	0	0	1	1

To, make it simple to remember just take the the input variable which occurs as '1' corresponding to the 1s in the output and add them all.

After constructing the truth table now prepare equation of the outputs. Since you have two outputs there will be two outputs.

$$Y_0 = \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0 + \bar{I}_3 \bar{I}_2 \bar{I}_1 I_0$$

We can use sop method to find out the Y₀ or Y₁ equation expression. However since we know that all other inputs will remain zero we can just ignore or discard those variables from the expression. So, the Y₀ will become

$$Y_0 = I_1 + I_3$$

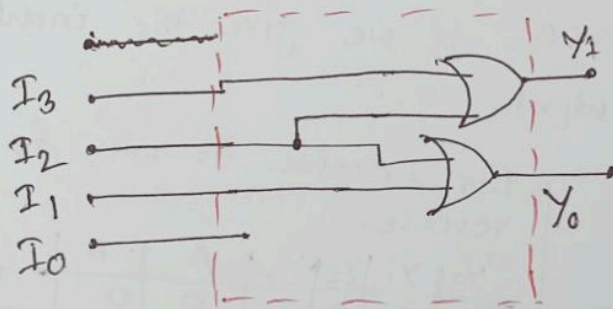
Similarly we can make Y₁ equation

$$Y_1 = \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0 + \bar{I}_3 \bar{I}_2 I_1 \bar{I}_0 = I_2 + I_3$$

we can see that from the expression of Y_0 and Y_1 the variable input I_0 is absent. It happens due to the logical derivation of the expression. However this input ~~is~~ may be named as the Idle input. Means the values on that input will not have any impact on the circuit.

For the expression we derived will have the below circuit

$$Y_0 = I_1 + I_3, \quad Y_1 = I_2 + I_3$$



Now, if we put $I_0 = 1$, $I_1 = 0$, $I_2 = 0$, $I_3 = 0$ which is our 1st combination the output will be $Y_1 = 0$, $Y_0 = 0$, which matches the truth table. As we do not have I_0 variable in the equation we need to keep it disconnected from the overall ~~it~~ circuit.

We can check for other combination also and will find the output of the circuit matches with the truth table.

Priority Encoder

Priority encoders implement the solution for ^{when} multiple inputs ~~become~~ are high. To prevent the multiple input becoming high, the priority encoder apply priority to the input. It means which input has higher priority than other input. We may have regular sequence priority and irregular sequence priority.

We need only one Rule to make the truth table.

* The variable or Input having higher priority of the subjected input will become Zero and the lower priority input will be in don't care condition.

Let's take a Regular Sequence

$I_0 > I_1 > I_2 > I_3 > I_4 > I_5 > I_6 > I_7$

Since these are don't care we can make them as zeros after constructing the input table using the rule

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	H_7	H_6	H_5	H_4	H_3	H_2	H_1	H_0	O_2	O_1	O_0
X	X	X	X	X	X	X	1	0	0	0	0	0	0	0	1	0	0	0
X	X	X	X	X	X	1	0	0	0	0	0	0	0	1	0	0	0	1
X	X	X	X	X	1	0	0	0	0	0	0	0	1	0	0	0	1	0
X	X	X	X	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0
X	X	X	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0
X	X	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	1

Priority Encoder Input. Priority encoder circuit Normal encoder output

Now find the Expression for Priority circuit output.

$$H_0 = I_0$$

$$H_1 = I_1 \bar{I}_0$$

$$H_2 = I_2 \bar{I}_1 \bar{I}_0$$

$$H_3 = I_3 \bar{I}_2 \bar{I}_1 \bar{I}_0$$

$$H_4 = I_4 \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0$$

$$H_5 = I_5 \bar{I}_4 \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0$$

$$H_6 = I_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0$$

$$H_7 = I_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 \bar{I}_2 \bar{I}_1 \bar{I}_0$$

Find the minterm in Priority encoders each output and prepare as "SOP minterm"

Remember Priority circuit output is not summed. It will be fed to the normal encoder input.

$$O_0 = I_1 + I_3 + I_5 + I_7$$

$$O_1 = I_2 + I_3 + I_6 + I_7$$

$$O_2 = I_4 + I_5 + I_6 + I_7$$

Normal Encoder output equations.

The process is same for irregular Sequence. More priority will be '0' less priority don't care.

We have a sequence like

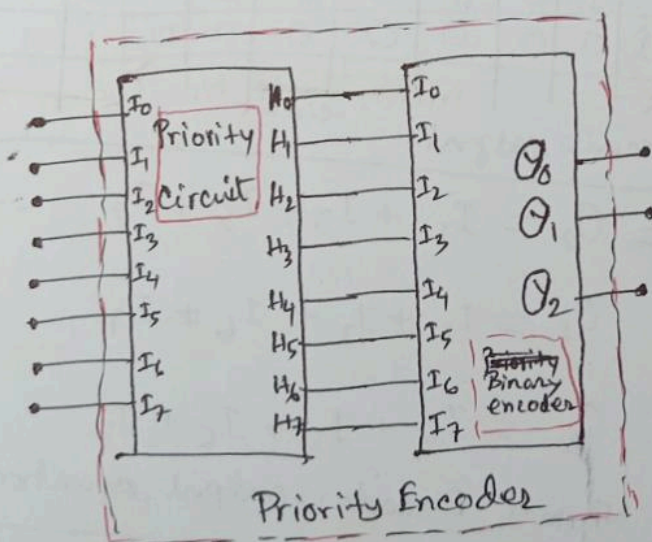
$I_0 > I_1 > I_5 > I_6 > I_7 > I_3 > I_2 > I_4$

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0
X	X	X	X	X	X	X	1
0	0	0	X	0	1	0	0

first take I_0 , it has most priority means greater of all so. It will be 1 and rest all will be don't care. I_1 is less than I_0 ^{only} so, I_0 will be zero, and all other will be don't care.

Let's take I_2 . For, I_2 is greater than I_4 so, I_4 will be don't care and all other will be zero.

→ After normalizing and determining the final output we can construct the diagram like below.



Remember Priority encoder is comprised of a priority circuit and a Binary Encoder!

Priority circuit normalize the initial inputs and make suitable to feed to the Binary encoder's operation.

Mux

Mux or multiplexers are used to make data stream or signals directed to a specific path. Mux has multiple input channel and only 1 output. So, we will need a mechanism by which we can select a particular data line to be connected to the output. Here comes the selector for the mux.

Let's take an example of a 2×1 mux where we will have 2 input and only 1 output. Importantly there will be a selector. let us ~~to~~ make the truth table for the whole combinations of selector, data and output.

we can take only 8 combinations, since it will repeat on further.

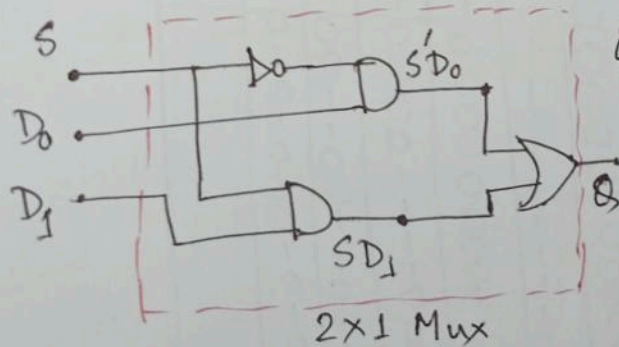
S	D ₁	D ₀	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

When the selector is '0' it will connect D₀ to the output.

When the selector is '1' it will connect the D₁ to the output.

Now we can use SOP ~~exp~~ to find the expression of Q.

Find the 1s in Q and make corresponding SOP expression taking all input variables.



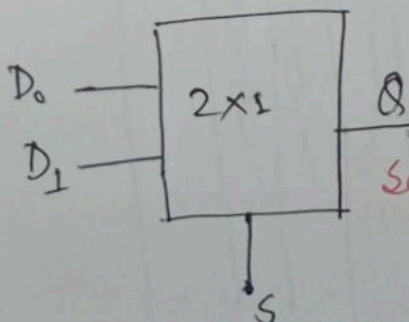
$$Q = \bar{S} \bar{D}_1 D_0 + \bar{S} D_1 D_0 + S D_1 \bar{D}_0 + S D_1 D_0$$

$$= \bar{S} \bar{D}_1 D_0 + \bar{S} D_1 D_0 + S D_1 (\bar{D}_0 + D_0)$$

$$= \bar{S} \bar{D}_1 D_0 + \bar{S} D_1 D_0 + S D_1$$

$$= \bar{S} D_0 (\bar{D}_1 + D_1) + S D_1$$

$$= \bar{S} D_0 + S D_1$$



Selector pins are placed at bottom.

Now, there would be a question? How many selectors we would require. It depends on your number of inputs. One selector can handle 2 input. two selector will handle 4 inputs. 3 selectors will manage $2^3 = 8$ inputs.

Now, let's say we have 4 data line or 4 data input. many selectors we will need. 2 selectors. Because 2 selectors will have 4 combination, upon which we can decide for which combination we will pass which data line. As we shown

Previously we can make the output expression constructing truth table for each combination which is

$Q = \bar{S}_1 \bar{D}_0 + S_1 D_1$
taken as 8 combinations since the 16 combination will be repetitive.

$$= \bar{S}_1 \bar{D}_0 + S_1 D_1 + \bar{S}_1 \bar{D}_0 + S_1 D_1 + \dots$$

$$= \bar{S}_1 \bar{D}_0 (D_1 + \bar{D}_1) \cdot (D_2 + \bar{D}_2) \cdot (D_3 + \bar{D}_3) \cdot (S_1 + \bar{S}_1)$$

$$= \bar{S}_1 \bar{D}_0 D_1 + S_1 D_0 \bar{D}_1$$

But for two Selector

the 32 com

total combination

including data line will be

$$2^6 = 64$$

which will require a big Truth table. And importantly it will not be repetitive like a single Selector.

S ₁	S ₀	D ₃	D ₂	D ₁	D ₀	Q
0	0	0	0	0	0	
0	0	0	0	0	1	
0	0	0	0	1	0	
0	0	0	0	1	1	
0	0	0	1	0	0	
0	0	0	1	0	1	
0	0	0	1	1	0	
0	0	0	1	1	1	
0	0	1	0	0	0	
0	0	1	0	0	1	
0	0	1	0	1	0	
0	0	1	0	1	1	
0	0	1	1	0	0	
0	0	1	1	0	1	
0	0	1	1	1	0	
0	0	1	1	1	1	
0	1	0	0	0	0	
0	1	0	0	0	1	
0	1	0	0	1	0	
0	1	0	0	1	1	
0	1	0	1	0	0	
0	1	0	1	0	1	
0	1	0	1	1	0	
0	1	0	1	1	1	
0	1	1	0	0	0	
0	1	1	0	0	1	
0	1	1	0	1	0	
0	1	1	0	1	1	
0	1	1	1	0	0	
0	1	1	1	0	1	
0	1	1	1	1	0	
0	1	1	1	1	1	
1	0	0	0	0	0	
1	0	0	0	0	1	
1	0	0	0	1	0	
1	0	0	0	1	1	
1	0	0	1	0	0	
1	0	0	1	0	1	
1	0	0	1	1	0	
1	0	0	1	1	1	
1	0	1	0	0	0	
1	0	1	0	0	1	
1	0	1	0	1	0	
1	0	1	0	1	1	
1	0	1	1	0	0	
1	0	1	1	0	1	
1	0	1	1	1	0	
1	0	1	1	1	1	
1	1	0	0	0	0	
1	1	0	0	0	1	
1	1	0	0	1	0	
1	1	0	0	1	1	
1	1	0	1	0	0	
1	1	0	1	0	1	
1	1	0	1	1	0	
1	1	0	1	1	1	
1	1	1	0	0	0	
1	1	1	0	0	1	
1	1	1	0	1	0	
1	1	1	0	1	1	
1	1	1	1	0	0	
1	1	1	1	0	1	
1	1	1	1	1	0	
1	1	1	1	1	1	

and so on

Complete TT of Single Selector.

repetitive

$$Q = S'_1 D'_1 D'_0 + S'_1 D'_1 D_0 + S'_1 D_1 D'_0 + S'_1 D_1 D_0 + S_1 D'_1 D'_0 + S_1 D'_1 D_0 + S_1 D_1 D'_0 + S_1 D_1 D_0$$

Some terms will repeat.

We can do. We can simplify the behavior of the total arrangements by statements. let's say we will have

$D_3 D_2 D_1$ & D_0 input and Q output. We will have two selectors S_1 & S_0 . so, we will have total 4 combination of S_1, S_0 . We will define which combination of S_1, S_0 will pass which Data to the output. In this way we can easily make the design

S_1	S_0	Q
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

for 0 0 it will pass D_0 to the output

" 0 1 " " " D_1 " " "

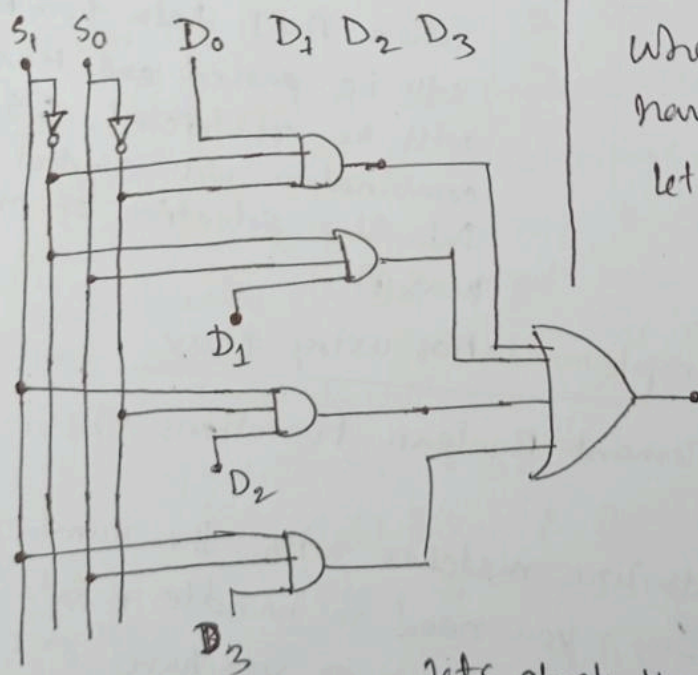
" 1 0 " " " D_2 " " "

" 1 1 " " " D_3 " " "

Expression of Q will be considering all data to be higher for the set combinations. so, Q will

$$Q = \bar{S}_1 \bar{S}_0 \cdot D_0 + \bar{S}_1 S_0 \cdot D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

now lets make the circuit.



let us verify our logic whether it is working fine or having some error.

let's say $S_1 = 1, S_0 = 0$ and

$$D_2 = 1$$

First we see for $S_1 S_0 = 10$ combination we will pass D_2 value to the Q .

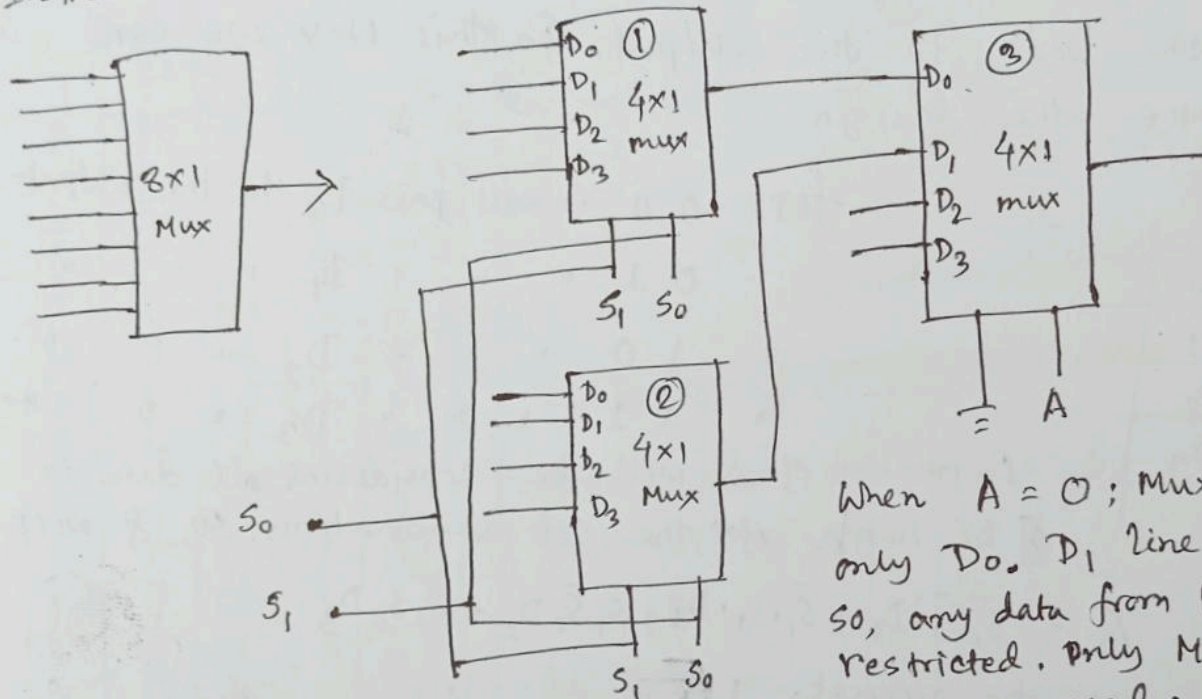
here $D_2 = 1$ so, the Q value should be 1

lets check the equation.

$$\begin{aligned} Q &= \bar{S}_1 \bar{S}_0 \cdot D_0 + \bar{S}_1 S_0 \cdot D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3 \\ &= \bar{1} \bar{0} \cdot (\text{any}) + \bar{1} \cdot 0 \cdot (\text{any}) + 1 \cdot \bar{0} \cdot 1 + 1 \cdot 0 \cdot (\text{any}) \\ &= 0 \cdot 1 \cdot \text{any} + 0 \cdot 0 \cdot \text{any} + 1 \cdot 1 \cdot 1 + 1 \cdot 0 \cdot \text{any} \\ &= 0 + 0 + 1 + 0 \\ &= 1 \text{ which is the value of } D_2 \text{ data.} \end{aligned}$$

Modularity

We may not always have suitable mux according to our requirement. Let's say we need to handle 8 data input but we may have 4x1 mux. Or say we need to handle 6 devices data line but we have 4x1 mux. In that case we can use multiple 4x1 mux to adjust our requirement.



When $A = 0$; Mux 3 will pass only D₀. D₁ line will be off so, any data from Mux ② will be restricted. Only Mux ① data will be passed. Similarly when $A = 1$ data from Mux ② will be passed and Mux ① data will be restricted. S₁ S₀ combination will control the Data line selection of Mux ① and Mux ②.

Function Implementation using Mux

Mux can be used to implement Boolean Functions. These can be of two cases.

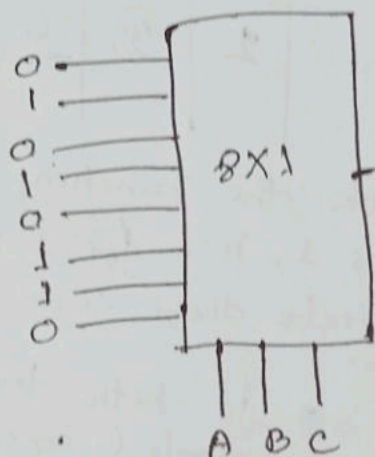
- ① When the required data line matches with the number of inputs of a mux. Say, you need to handle 8 data line and you are asked to use 8x1 mux or you have 8x1 mux in your hand. n variable function can be implemented by $2^n \times 1$ mux. Let's say 3 variable function will have $2^3 = 8$ combinations requiring 8 data line. So, it can be implemented directly using 8x1 mux.

Function using Mux $F(A, B, C) = \sum(1, 3, 5, 6)$

Let's say we have a Boolean function like below Truth table.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

here we have 3 variables and 8 combinations.
So, we can directly use $2^3 = 8 \times 1$ mux.



8×1 mux have
3 selector pins.

data will be passed for each of the selector pin combinations. Means the combination of the variables are provided by the combination of the selector pins.

So, we need to check that whether we can match the number of function variables are matched with the available selector pins.

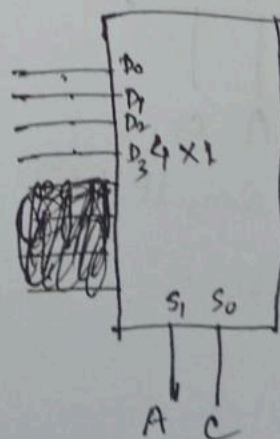
Case 2

Let's say we have n variable function and asked to implement using a mux which has less than 2^n input or less the n selector pin.

Let's have 3 variable function to be implemented by 4×1 mux. 3 variable function will have $2^3 = 8$ combinations. 4×1 mux will have only 2 selector pins and will give only 4 combinations. So, mismatch in variables and selector pins. In that case we need to make some data line as the variable input line. For this we need to perform a methodological operation as follows. Let's implement the same function above using 4×1 mux. * First take two variables for two selector pins of 4×1 mux

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

① Let's take A & C will be used in two selector pins of 4×1 mux



② Now we need to find how we can use the variable B in the data lines.

For that we need to construct a truth table

	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

* Find the terms where $B=0$ and write in $\bar{B}$ Row
 * Find the terms where $B=1$ and write in B Row

Variable form

	D_0	D_1	D_2	D_3
$\bar{B}$	0	1	4	5
B	2	3	6	7

$B=0 \rightarrow \bar{B}$ Row
 $B=1 \rightarrow B$ Row

* Now check the Function; for which terms the function is 1. here for, 1, 3, 5 and 6 the function is 1. Circle these term in the special table.

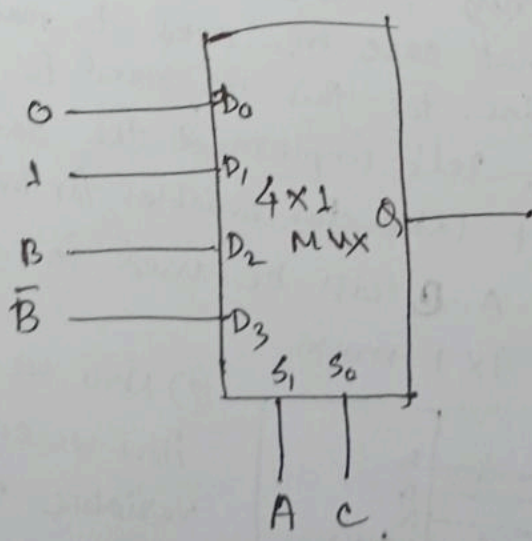
	D_0	D_1	D_2	D_3
$\bar{B}$	0	1	4	5
B	2	3	6	7
	0	1	B	$\bar{B}$

* if both term in a column gets circled consider the data line as 1

* if only one term is circled make the data line with the form it get's from the variable. Here 6 is taking the form B . And 5 is taking the form $\bar{B}$.

* lastly if there is no circle, make the data line 0.

Now, Draw the whole solution using 4×1 mux.



you have done it!! 🎉